

Voice-Driven PowerPoint Editing Agent

SURF Project Experimental Report

July 2026

Abstract

This project adopts PPTPilot, the execution agent introduced in the PPTArena paper, to investigate executable slide editing from spoken instructions. The work comprised local adaptation of the released code, reproduction of the graphical user interface (GUI), refactoring into a command-line interface (CLI), integration of two automatic speech recognition (ASR) systems, and end-to-end experimentation. The resulting system preserves speech transcripts, agent execution logs, edited PPTX files, and verification results.

The experiments used two transcription models, faster-whisper large-v3-turbo and Qwen3-ASR-1.7B. Four slide-editing tasks were selected from the speech corpus prepared by the data team; each task contained 16 combinations of spoken formulation and acoustic condition, yielding 64 audio recordings in total. Text correction, title font size, and slide-number insertion were evaluated using deterministic rules, whereas image-layout results required both object-geometry checks and rendered-slide inspection. In the first editing pass, Qwen3-ASR increased accuracy on the three rule-based tasks from 66.67% to 79.17% relative to faster-whisper. Both ASR systems achieved 10/16 on the image-layout task, indicating no improvement from changing the transcription model. For the id31 horizontal image-centering task, 64 slide layouts produced by the two ASR systems after the first and second editing passes were further examined; 37 (57.8%) passed geometric and visual verification, with overlapping images constituting the dominant failure mode.

The results show that a stronger ASR system can reduce the loss of critical values and units, but accurate transcription alone does not guarantee correct editing. Object identification, generated-code reliability, and output verification are also decisive for end-to-end success. The principal outcome of the project is an operational and traceable experimental pipeline for voice-driven slide editing with layered verification.

1 Project Objectives

The project was motivated by the question of how hesitations, pauses, self-corrections, informal expressions, and acoustic conditions in spoken instructions affect structured slide editing. The investigation concerns not only whether speech can be transcribed, but also whether the transcript preserves the editing action, target object, attribute, and value, and whether the agent can execute that information correctly. This report focuses on the implemented system and completed experiments, and addresses the following questions:

1. Do different spoken formulations and acoustic conditions affect the preservation of task-critical information and downstream editing outcomes?
2. Do end-to-end failures arise primarily in ASR transcription, agent-level object and layout planning, or generated-code execution?
3. Can PPTPilot be reproduced locally and adapted into a batch-executable, traceable, and verifiable prototype for voice-driven PowerPoint editing?

The objective was not to develop a complete product, but to establish a reproducible experimental pipeline and to characterize its capabilities and limitations through actual outputs.

2 Technical and Data Foundations

2.1 Execution Agent: PPTPilot from PPTArena

PPTArena is a benchmark for in-place editing of real PowerPoint files. PPTPilot is the slide-editing agent introduced, with released source code, in *PPTArena: A Benchmark for Agentic PowerPoint Editing*. This project used PPTPilot as its execution core, first reproducing the original GUI and editing framework, and subsequently implementing local fixes, CLI refactoring, and a speech-input extension.

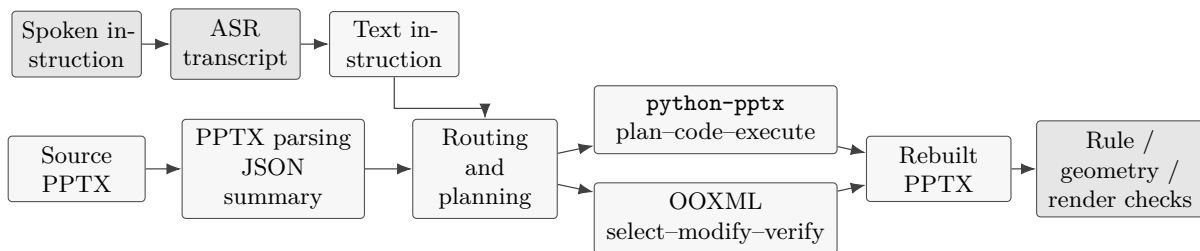


Figure 1: PPTPilot’s core editing workflow and the speech-input and verification stages added in this project. Shaded nodes denote project extensions.

At execution time, the system first parses a PPTX file into a JSON summary containing slides, objects, text, coordinates, and style information. A routing model then selects the execution path. Standard formatting and batch object operations are performed through `python-pptx`, whereas complex structural or low-level style changes are implemented by directly modifying and repackaging OOXML. Beyond the original framework, this project added ASR, execution tracing, and independent output verification.

2.2 Data Source and Experimental Subset

The tasks were drawn from TSBench, introduced in the *Talk to Your Slides* paper. TSBench Original contains 379 slide-editing instructions. For each instruction, the data team prepared four spoken formulations and rendered each formulation under four acoustic conditions—clean, bandlimit, noise, and reverb—producing $379 \times 4 \times 4 = 6,064$ WAV files. The resulting corpus was delivered as SSFT260706_1 and SSFT260706_2.

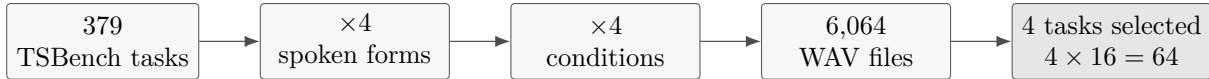


Figure 2: Corpus expansion procedure and the experimental scope of this report.

Four representative editing tasks were selected from the full audio collection: id5 (text correction), id20 (title font size), id31 (horizontal image centering), and id47 (slide-number insertion in the lower-right corner). Each task covered four spoken variants and four acoustic conditions, yielding 64 audio inputs. The corresponding PPTX files and editing instructions originated from TSBench, while the audio variants were produced by the data team.

3 Development Process and Stage Outputs

Table 1: Project stages and completed work

Stage	Work completed	Principal output (name and role)
Source-code adaptation	Fixed absolute paths, model configuration, API calls, and output-directory handling	PPTArena-main : locally repaired baseline
GUI reproduction	Reproduced the paper’s interaction workflow; added model selection, error handling, and result persistence	PPTArena_GUI : reproduced GUI version
CLI refactoring	Removed Flask, front-end, session, and deployment modules; added single-case and batch entry points	PPTArena_CLI : CLI batch-processing version
Speech extension	Integrated faster-whisper and Qwen3-ASR, preserved transcripts, and connected transcription to editing	PPTAgent : first-stage speech-enabled version
Final experiments	Replaced the planning and editing models with DeepSeek variants; ran the speech subset and added output verification	Agent : final experimental version

The core codebase decreased from approximately 6,298 lines in the GUI stage to 3,276 lines in the CLI stage and 2,468 lines in the final agent. This reduction primarily resulted from removing the Web interaction, session, and deployment layers. Core capabilities—including PowerPoint parsing, planning, and the dual OOXML/`python-pptx` editing paths—were retained; the lower line count was not achieved by removing editing functionality.

3.1 Stage-by-Stage Evolution

The successive versions formed a progressive development sequence. The initial GUI reproduction made it possible to observe PPTPilot’s routing, editing, and preview processes and to confirm that the core framework operated locally. As the number of test cases increased, Web interaction, session state, and manual operations became constraints on batch experimentation. The system was therefore refactored into a CLI with explicit control over inputs, models, editing passes, logs, and output directories.

On top of the CLI, two ASR systems were added as the speech-input layer. Transcripts, agent logs, and edited PPTX files were stored separately, enabling ASR information loss to be distinguished from agent execution errors. Rule-based, geometric, and rendered-slide verification were then introduced, advancing the system from simple executability to batch comparison and failure localization.

3.2 Final System

The final system places ASR before PPTPilot’s text-instruction interface and adds independent rule-based, geometric, and rendered-slide verification at the output stage. Complex structural modifications use OOXML, while simpler or repetitive operations use `python-pptx`. In the final configuration, DeepSeek V4 Flash performs planning, and DeepSeek V4 Pro generates editing code or XML modifications.

The unified `run.py` entry point supports individual audio files, directory-level batch processing, ASR selection, configurable editing passes, and reuse of existing transcripts. Transcripts, edited PPTX files, and logs are stored separately to support reproduction and error localization.

4 Experimental Design

4.1 Data

The complete audio collection contains 6,064 mono WAV files sampled at 24 kHz, with a total duration of approximately 13.87 hours. It combines 379 instructions, four spoken variants, and four acoustic conditions.

The final experiment selected four PowerPoint editing tasks from the data organized by TSBench task identifier. Each task contained four spoken variants under four acoustic conditions, for 16 recordings per task and 64 recordings overall. The same audio files were processed by two ASR systems:

- ASR-2: faster-whisper large-v3-turbo;
- ASR-6: Qwen3-ASR-1.7B.

4.2 Tasks

Table 2: The four tasks used in the final experiment

Task	Target edit	Principal capability	Verification method
id5	Correct spelling, grammar, and typographic errors	Text modification	Text rules
id20	Set all titles to 32 pt	Numeric slots, object identification, batch formatting	Title-size rules
id31	Arrange images horizontally and center the group	Multi-object spatial layout	Geometry and rendering checks
id47	Add slide numbers in the lower-right corner	Structural insertion and positional constraints	Text and coordinate rules

These four tasks form a progression from simple text correction to precise formatting, structural insertion, and spatial layout. They constitute the experimental subset used to validate the current system pipeline and do not represent the full task coverage of TSBench.

4.3 Evaluation Criteria

The outputs were evaluated at four levels:

1. **Readable output:** whether an openable PPTX file was generated;
2. **Edit signal:** whether the output differed from the input;
3. **Rule correctness:** whether the target text, font-size, or slide-number constraints were satisfied;
4. **Geometric correctness:** whether objects overlapped, exceeded slide boundaries, or were incorrectly positioned.

Accordingly, a readable output indicates only that file generation completed. An editing task was counted as successful only when its target text, formatting, or geometric constraints were also satisfied.

5 Experimental Results

5.1 Overall First-Pass Results

Table 3: First-pass end-to-end results for the two ASR systems

Metric	ASR-2	ASR-6	Difference / pp
Readable final PPTX generated	61/64 (95.31%)	63/64 (98.44%)	+3.13
Output with an edit signal	52/64 (81.25%)	60/64 (93.75%)	+12.50
Correct under rule-based verification (id5, id20, id47)	32/48 (66.67%)	38/48 (79.17%)	+12.50
Correct under geometric and visual verification (id31)	10/16 (62.50%)	10/16 (62.50%)	0.00
Correct under task-specific verification	42/64 (65.63%)	48/64 (75.00%)	+9.37

The 48 rule-evaluated cases comprise id5, id20, and id47, for which deterministic checks were available. The 16 first-pass id31 outputs were evaluated separately using geometric measurements and rendered-slide inspection. ASR-6 improved both output rate and accuracy on the three rule-based tasks, but overall task-specific accuracy remained 75.00%. Thus, a readable PPTX and an editing log establish only that the pipeline ran; the agent may still make substantive errors in object identification, planning, or execution.

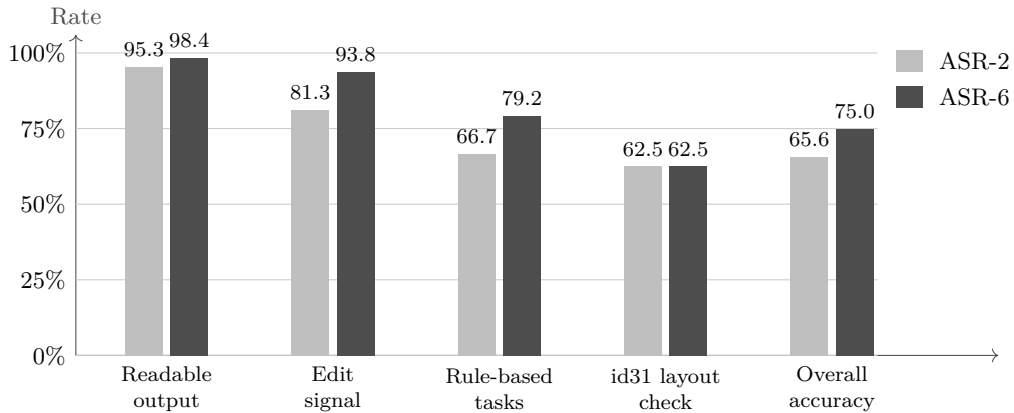


Figure 3: First-pass end-to-end metrics for the two ASR systems. Qwen3-ASR improved readable output, edit signal, rule-based task performance, and overall accuracy, while id31 remained unchanged.

5.2 Task-Level Results

Table 4: First-pass verification results for the four tasks

Task	ASR-2	ASR-6	Interpretation
id5 spelling and grammar	16/16	16/16	Relatively stable
id20 title size (32 pt)	2/16	7/16	Principal bottleneck
id31 horizontal image centering	10/16	10/16	Layout planning remains unstable
id47 lower-right slide numbers	14/16	15/16	Generally reliable
Total	42/64 (65.63%)	48/64 (75.00%)	ASR-6 higher by 9.37 pp

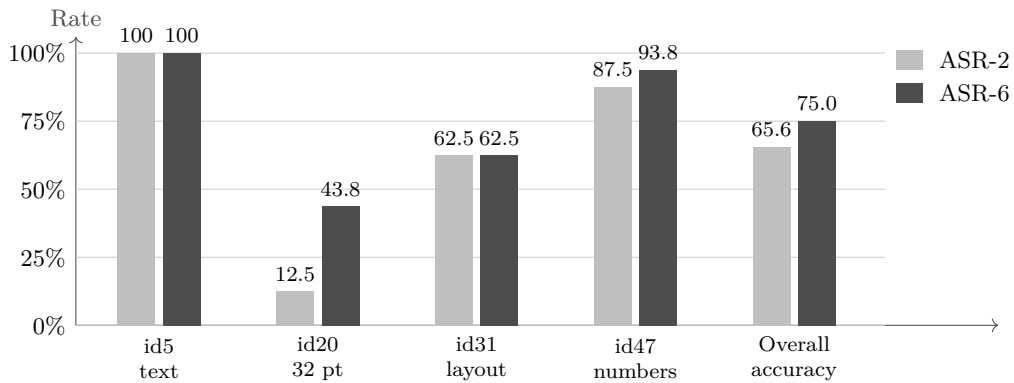


Figure 4: First-pass accuracy for the four tasks. Id31 was evaluated geometrically and visually; the remaining tasks used deterministic rules.

Id5 achieved 100% under both ASR systems, indicating that simple text correction was already relatively stable. Id20 performed worst, demonstrating persistent difficulty with exact numeric values, title-object identification, and formatting changes. Id31 was assessed using geometry and rendered slides, whereas the other three tasks used deterministic rules. The total therefore reports first-pass correctness under the task-appropriate verification method.

5.3 Targeted Second-Pass Experiment

After the first pass, two poorly performing task groups were selected for a second editing pass: the 16 id20 cases under ASR-2 and the 16 id31 cases under each of ASR-2 and ASR-6, for 48 matched cases in total. The reruns reused the same transcripts; the second pass replanned and executed against the current PowerPoint state. Here, “targeted” means that difficult task groups were selected for additional runs, not that a verifier triggered retries only for first-pass failures.

Table 5: Targeted second pass: final states of matched cases after one and two editing passes

Task / ASR	First-pass readable	Second-pass readable	First-pass correct	Second-pass correct	Δ / pp
id20 / ASR-2	16/16	16/16	2/16 (12.50%)	3/16 (18.75%)	+6.25
id31 / ASR-2	14/16	15/16	10/16 (62.50%)	8/16 (50.00%)	-12.50
id31 / ASR-6	15/16	13/16	10/16 (62.50%)	9/16 (56.25%)	-6.25
Total	45/48 (93.75%)	44/48 (91.67%)	22/48 (45.83%)	20/48 (41.67%)	-4.17

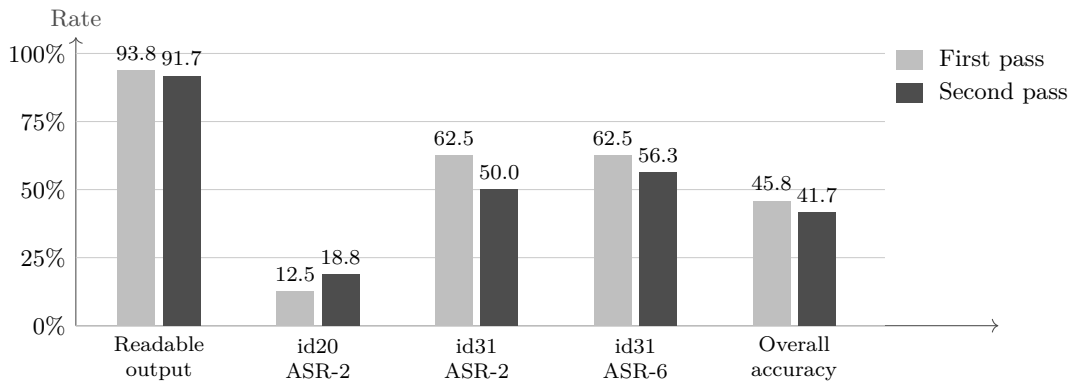


Figure 5: Final results after one and two passes in the targeted reruns. Accuracy was determined by the id20 rule-based verifier and the id31 geometric/rendered-slide verifier.

Case-level comparison reveals changes concealed by aggregate counts. For id20, one case remained correct, one changed from correct to incorrect, and two changed from incorrect to correct, yielding a net gain of one. Among the 32 id31 cases, 10 of the 20 first-pass successes became incorrect after the second pass, while only 7 of the 12 first-pass failures were repaired, producing a net loss of three. Overall correctness therefore declined from 22/48 to 20/48 not because the second pass was wholly ineffective, but because it damaged some previously correct results while repairing a subset of failures.

The additional pass also increased computational and execution cost. Automatic retry events recorded in the corresponding logs rose from 3 in the first-pass runs to 15 in the two-pass runs, while readable outputs decreased from 45/48 to 44/48. Full replanning and re-execution therefore introduced new code-generation and layout risks. A preferable strategy would preserve the first-pass output and use rule-based or geometric verifiers to trigger targeted retries only for failed cases.

5.4 Critical Slot: 32 pt

Post hoc inspection of the id20 transcripts yielded the following results:

Table 6: Preservation of the id20 critical slot and downstream correctness

Metric	ASR-2	ASR-6
Transcript fully preserved “32 pt”	3/16	7/16
Correct under downstream rules	2/16	7/16

ASR-6’s improvement arose primarily from better preservation of the critical value and unit. However, one ASR-2 transcript retained “32 pt” while the downstream edit still failed. Accurate ASR is therefore one necessary condition for end-to-end success, but it is not sufficient.

5.5 Image-Centering Task: Geometric and Visual Inspection

Id31 requires the three existing images to be arranged horizontally and centered as a group. An edit signal alone cannot determine whether this layout is correct; consequently, the task was evaluated using both PPTX object geometry and rendered-slide inspection. The 64 final states produced by two ASR systems, two editing passes, and 16 spoken/acoustic combinations were classified as pass, image overlap, positional error, or missing output.

Table 7: Verification results for final id31 states

ASR	Pass	Correct	Overlap	Offset	No output	Success rate
ASR-2	First	10/16	3	1	2	62.5%
ASR-2	Second	8/16	7	0	1	50.0%
ASR-6	First	10/16	3	2	1	62.5%
ASR-6	Second	9/16	4	0	3	56.3%
Total	—	37/64	17	3	7	57.8%

All 57 generated PPTX files retained the three original images. The failures did not result from deleted media; rather, the agent often aligned multiple images to the same page center. Overlap occurred in 17 cases: two images overlapped in 14 cases, and all three overlapped in 3. A further 3 cases were misplaced, and 7 produced no output.



(a) Pass: three images distributed at equal spacing (b) Failure: all three images overlap at the center

Figure 6: Actual id31 outputs. Left: a passing result. Right: the three original images overlap at the slide center, leaving only the top layer visible.

ASR-2 and ASR-6 passed 18/32 and 19/32 cases, respectively, a difference of only 3.1 percentage points. By editing pass, the corresponding totals were 20/32 after the first pass and 17/32 after the second, a decline of 9.4 percentage points. The four acoustic conditions yielded 9/16, 12/16, 7/16, and 9/16 passes, with no stable pattern. The principal bottleneck therefore lies in the agent’s spatial reasoning and execution, and targeted retries should be triggered by geometric verification.

6 Failures, Challenges, and Improvements

6.1 Localization of Major Failures

Table 8: Major failure stages and supporting evidence

Failure stage	Experimental evidence	Principal mechanism
Loss of critical ASR information	For id20, complete preservation of “32 pt” increased from 3/16 to 7/16, while downstream correctness increased from 2/16 to 7/16	Errors in numeric values or units directly alter editing parameters
Agent object and layout planning	All 57 existing id31 PPTX files retained three images, yet 17 exhibited overlap and 3 exhibited misplacement	“Center the arrangement horizontally” was interpreted as centering each image independently, without a global layout constraint
Generated code and API calls	In the 64 first-pass runs, generated-script errors triggered retries in 6 cases	Incorrect access to <code>runs[0]</code> in empty paragraphs and misuse of color APIs, media enumeration, private placeholder attributes, or slide-size properties
Loop control and output verification	Id31 correctness declined from 20/32 after one pass to 17/32 after two passes, while overlap increased	Full replanning in the second pass can overwrite correct first-pass results; the system lacks failure-triggered execution, rollback, and state comparison

These failures cannot be attributed uniformly to ASR. In id20, for example, one ASR-2 transcript preserved “32 pt” but the downstream edit still failed, showing that the agent may select the wrong object or generate faulty code even when transcription is adequate. Conversely, id31 achieved 10/16 first-pass successes under both ASR systems, and their two-pass totals differed by only 3.1 percentage points. The principal bottleneck for this task therefore lies in object localization and spatial planning.

Stronger planning or code-generation models may reduce syntax errors, API hallucinations, and some object-selection errors, but model capability is only one component. If the system continues to rely on unconstrained generation of low-level API code without object inventories, type constraints, output verification, or rollback, structural errors such as overlap, misplacement, and corruption of existing formatting may persist. Future work should compare agent models while holding the ASR system, tasks, and verifiers fixed, thereby separating the contribution of model capability from that of tool constraints.

6.2 Engineering Challenges and Current Mitigations

Table 9: Engineering challenges, current mitigations, and unresolved issues

Challenge	Mitigation implemented in this project	Remaining limitation
Dependence on the authors’ environment	Replaced absolute paths and unified input, output, and model configuration	Complete environment locking and one-command deployment remain unavailable
GUI unsuitable for batch execution	Refactored the CLI and added entry points for single tasks, directory batches, and transcript reuse	Large-scale checkpoint recovery and configuration manifests require further development
Model cost and availability	Adopted affordable DeepSeek models for planning and editing	No systematic capability–cost–latency curve has yet been established
Entangled ASR and agent errors	Preserved transcripts, logs, and outputs, and inspected critical slots	Comprehensive slot-level metrics and a structured intent layer remain absent
Inconsistent evaluation criteria	Defined rules for text, font size, and slide numbers, and added geometry/rendering checks for layout	Unified verifiers covering more tasks and systematic human auditing are still needed

6.3 Directions for Further Improvement

1. Introduce a structured intent layer between ASR and the agent that explicitly records actions, target objects, attributes, values, units, and coreference relations, and validates critical slots before execution;
2. Replace unconstrained low-level API generation with typed editing tools and explicit object inventories that restrict callable functions, parameter ranges, and object scope;
3. Establish a planning–execution–verification loop in which rules, geometric differences, and rendered outputs trigger targeted retries, while correct first-pass results and rollback checkpoints are preserved;
4. Extend evaluation to additional TSBench tasks, ASR systems, and editing agents; include a clean-text upper bound, unify failure categories, and report confidence intervals and significance tests.

7 Research Motivation and Stage-Level Findings

7.1 Original Motivation and Scope of This Stage

The project originally examined how large language models could interpret spoken instructions and execute slide edits. Hesitations, pauses, self-corrections, and informal phrasing, together with acoustic variations such as band limitation, noise, and reverberation, may cause actions, target objects, attributes, or values to be lost during transcription. The original research design therefore extended beyond plain speech-to-text conversion: it also envisioned structured representations of editing intent and comparison of downstream behavior between speech-based input and transcript-only input.

This report completes a transcript-mediated system validation within that broader agenda:

two ASR systems process the same speech inputs, PPTPilot executes the resulting instructions, and transcripts, logs, PPTX files, and verification results are retained. The current data are insufficient to estimate a stable independent effect for any single acoustic condition, but they do distinguish two failure classes: loss of critical information during transcription and agent execution failure despite an adequate transcript.

7.2 Findings Supported at This Stage

- Transcription quality affects editing performance, but its impact is concentrated in task-critical slots. When Qwen3-ASR preserved “32 pt” more reliably, id20 correctness increased from 2/16 to 7/16;
- A high readable-output rate does not imply a high task-completion rate. ASR-6 produced readable first-pass outputs in 98.44% of cases, but overall task-specific verification accuracy was 75.00%;
- Correct transcription is not sufficient for task success. Even when critical information is preserved, the agent can fail through object selection, spatial planning, or code execution;
- Rendered-slide verification is indispensable. Failed id31 PPTX files still contained all three images; complete occlusion could be detected only by examining object geometry and slide renderings;
- A fixed increase in editing passes does not guarantee improvement. Verifier-triggered targeted retries are more appropriate than unconditional repetition.

7.3 Limitations of the Conclusions

The current experiment covers only four task types, two ASR systems, and one PowerPoint editing agent. It does not include a clean-text upper bound, structured intent representations, speech-native models, or complete statistical testing. The results therefore do not represent the full Speech2SlideEdit/TSBench benchmark, do not establish a stable monotonic effect for any acoustic condition, and do not justify claims of statistically significant model superiority. Their value lies in the reproducible system pipeline, traceable failure evidence, and evaluation framework established for subsequent full-scale experiments.

8 Conclusion

Starting from PPTPilot as introduced in the PPTArena paper, this project completed local adaptation, GUI reproduction, CLI refactoring, dual-ASR integration, and speech-driven editing experiments. The final system preserves audio transcripts, agent execution traces, edited PPTX files, and rule-based, geometric, and rendered-slide verification results, forming a complete evidence chain from input to failure analysis.

The results exhibit a clear hierarchy. At the pipeline level, the two ASR systems produced 61/64 and 63/64 readable first-pass outputs. At the semantic level, the stronger ASR system increased accuracy on the three rule-based tasks from 66.67% to 79.17% by preserving critical

values and units. At the execution level, id31 achieved only 10/16 first-pass successes under each ASR system; its combined two-pass success rate was 57.8%, with second-pass performance lower overall than first-pass performance. ASR improvements can therefore reduce upstream information loss, but agent object understanding, code reliability, and output control remain the principal limitations of the end-to-end system.

Across the 48 matched cases comprising id20 under ASR-2 and id31 under both ASR systems, final correctness declined from 22/48 after one pass to 20/48 after two, while automatic retry events recorded in the logs increased from 3 to 15. The second pass both repaired failures and overwrote correct results; it should therefore not be adopted as a default strategy.

Future work should jointly develop representations of task-critical information, constrained editing tools, geometric and rendered-slide verifiers, and rollback-capable targeted retry mechanisms.

References

- [1] Michael Ofengenden, Yunze Man, Ziqi Pang, Yu-Xiong Wang. *PPTArena: A Benchmark for Agentic PowerPoint Editing*. arXiv:2512.03042v2, 2025.
- [2] Kyudan Jung et al. *Talk to Your Slides: High-Efficiency Slide Editing via Language-Driven Structured Data Manipulation*. arXiv:2505.11604v4, 2026.
- [3] SURF-2026-0425 project description and internal experimental materials.
- [4] Project source code, stage reports, experimental outputs, and audit records, June–July 2026.

A Index of Principal Outputs

Output	Role	Description
PPTArena-main	Original baseline	Released paper code and starting point for local repairs
PPTArena_GUI	GUI reproduction	Local preview, model selection, error handling, and result persistence
PPTArena_CLI	Batch-processing foundation	Single-case and batch entry points without Web dependencies
Early GUI/CLI result comparison	Exploratory experiment	Inputs, outputs, and GUI/CLI records for 10 cases selected from the PPTArena dataset; not a controlled paired comparison
PPTAgent	First speech extension	Dual ASR integration, task mapping, and transcript persistence
Agent	Final experimental version	Unified entry point, DeepSeek models, four-task outputs, rule/geometric verification, case-level audit, and final report
SSFT260706_1 / _2	Audio collection	Four spoken variants under clean, bandlimit, noise, and reverb conditions